Verified Discovery: An Assurance Architecture for LLM-Mediated Mathematical Research

Sze Chun Yiu
Stockholm University
sze-chun.yiu@fysik.su.se

11 August 2026

Abstract

Large language models can solve difficult mathematical exercises and increasingly operate inside formal proof systems, yet exercise solving is not the same task as mathematical research. RAKL v3 adds persistent external task experience, versioned procedural lessons and experience-conditioned routing, creating a new assurance question: how can a system learn which mathematical moves to try without allowing past success, repeated failure or model confidence to become mathematical authority? We present a mathematical-research assurance layer that separates five coordinates: specification alignment, theorem truth, novelty, research value and verifier trust. Proof attempts are preserved as immutable task episodes; failed search remains distinct from impossibility; reusable proof lessons and strategy motifs are proposal-side abstractions; and experience may reprioritize proof routes but cannot promote theorem truth. Formal proofs remain bound to exact statement hashes and audited for axioms and verifier identity, while novelty remains a bounded, cutoff-scoped and defeasible certificate. Under a sound checker and fail-closed update rule, generator hallucinations or experience-derived routing cannot by themselves promote a false theorem. Verified lemma checkpointing converts many local generator errors into rejected branches and search cost, subject to specification and verifier-trust assumptions. The result is a v3 specialization for auditable mathematical discovery, not an autonomous-mathematician claim, a proof that repeated search failure implies impossibility, or evidence of universal discovery superiority.

1 The gap between solving and research

A useful caricature of an LLM mathematical solver is a conditional generator

$$G_\theta(s_t) \to a_t,$$

where the current state s_t contains a problem, partial derivation or proof state and a_t is a plausible next move. This capability can be extremely strong. AlphaProof demonstrates that model-guided search inside Lean can solve formalized olympiad and Putnam-level problems at a level far beyond earlier systems [2]. Other systems use LLM generation together with executable evaluation or evolutionary search to discover new constructions and algorithms [1, 3, 4].

Mathematical research adds obligations absent from a benchmark with a known target answer.

Recent work makes the same frontier distinction from complementary directions. Jiang et al. argue that formal-mathematics systems must move from predefined problem solving toward open-ended research agents with explicit support for exploration, abstraction and human–AI collaboration [8]. MA-ProofBench, meanwhile, evaluates 200 formalized mathematical-analysis theorems produced through a human-led, LLM-assisted formalization pipeline with independent expert review and reports substantial difficulty even for current models [9]. These results

strengthen the need to separate proof search, specification fidelity and research-level assurance rather than treating benchmark proof success as an end-to-end research certificate. A research agent must decide what is worth proving, survive long branches of failed ideas, distinguish experimental support from proof, ensure that a formal statement matches the intended theorem, identify rediscoveries under different notation, and make a novelty claim whose evidence is necessarily incomplete. A single scalar such as "mathematical confidence" is therefore the wrong abstraction.

The frequently invoked long-horizon calculation

$$0.99^{100} \approx 0.366, \qquad 0.99^{1000} \approx 4.3 \times 10^{-5}$$

is an appropriate warning for an unchecked chain if local errors can survive into the final answer. It is not the right end-to-end model once every admitted lemma is independently machine checked. In a proof-producing architecture, bad generations can increase search cost without becoming accepted logical premises. This distinction motivates the present work.

2 Five independent authority coordinates

Definition 1 (Mathematical authority state)**.** *For a mathematical claim c, define*

$$\alpha_{\mathrm{math}}(c) = (A_{\mathrm{spec}}, A_{\mathrm{truth}}, A_{\mathrm{novel}}, A_{\mathrm{value}}, A_{\mathrm{trust}}),$$

where the coordinates represent, respectively, informal–formal specification alignment, logical proof authority, novelty evidence, research-value assessment, and trust in the proof-checking path.

These coordinates form a product of scoped partial orders, not a total score. In particular,

$$A_{\mathrm{truth}} \not\Rightarrow A_{\mathrm{novel}}, \quad A_{\mathrm{novel}} \not\Rightarrow A_{\mathrm{truth}}, \quad A_{\mathrm{value}} \not\Rightarrow A_{\mathrm{truth}}.$$

A fully verified proof of a classical theorem is high on truth and low on novelty. An original-looking conjecture can be high on prospective value but have no truth authority. Collapsing the coordinates makes these cases indistinguishable.

3 Typed research state and persistent proof DAG

A scoped mathematical research program is represented by

$$\mathfrak{M}_t = (I_t, F_t, D_t, X_t, P_t, V_t, N_t, Q_t, H_t^-),$$

with frozen informal targets I_t, formal statements and alignment witnesses F_t, a dependency DAG D_t, counterexamples and refutations X_t, candidate proof artifacts P_t, verification receipts V_t, novelty dossiers N_t, research-value assessments Q_t, and immutable negative history H_t^-.

The dependency object D_t replaces the monolithic research transcript. Nodes can be definitions, conjectures, lemmas, representations, counterexamples, proof obligations, imported theorems or computational experiments. Edges record typed relations such as *implies*, *reduces to*, *refutes*, *specializes*, *generalizes* and *requires*. A verified lemma becomes a persistent checkpoint. A failed branch remains addressable negative history rather than being overwritten by the next fluent derivation.

4 Experience-governed mathematical search in RAKL v3

The v3 substrate distinguishes the mathematical research state from the system's accumulated task experience. A consequential conjecture test, representation change, tactic sequence, proof attempt or counterexample search can be frozen as a task episode containing the problem signature, relevant fibre snapshot, operators, action trace, observations, verification events, outcome, residual, provenance and cost. The episode records what happened; it does not certify why it happened.

A reusable mathematical lesson is a separate versioned object. It may encode a trigger such as a proof-state pattern, a scoped action such as changing representation or attempting a lemma family, expected effects, boundaries, supporting and contradicting episodes, a falsifier and validation obligations. Reflection can propose such a lesson, but proof-backed or fresh-transfer evidence is required before it is treated as reusable method state. The source episodes remain immutable even when a later lesson supersedes an earlier abstraction.

Experience may also alter the order in which admissible proof operators or paths are attempted. Let ρ_t be a routing policy derived from previous episodes. Then the proposal process may be written schematically as

$$a_t \sim G_\theta(\mathfrak{M}_t, E_t, \rho_t),$$

where E_t is the episode ledger. This changes search, not truth authority.

Proposition 1 (Experience cannot promote theorem truth)**.** *Assume theorem promotion requires an accepted proof artifact for the exact registered formal statement under an admitted verifier-trust profile. Then no update consisting only of task-episode storage, lesson consolidation or experience-conditioned routing can promote an unproved theorem.*

Proof. Episode, lesson and routing updates have codomain in experience or search-policy state. By assumption, theorem promotion has an additional necessary premise: an accepted proof artifact bound to the registered statement and trust profile. Therefore experience can change which proof attempts are generated or prioritized but cannot satisfy the missing theorem-promotion premise by itself. □

4.1 Failed search is not impossibility

A failed proof attempt creates negative search history. It does not establish that the theorem is false, that no proof exists in the registered logic, or that a required operator is missing. A counterexample may refute a universal statement; a machine-checked impossibility theorem may establish a scoped negative result; ordinary search failure establishes neither. V3 therefore retains the ladder

$$\text{failed episode} \to \text{candidate diagnosis} \to \text{discriminating evidence} \to \text{verified boundary lesson},$$

with no shortcut from repeated frustration to mathematical authority.

4.2 Proof strategy motifs remain lossy abstractions

Repeated successful operator sequences may nominate reusable strategy motifs. Such motifs can be valuable for retrieval and planning, especially in long proof DAGs with recurring bottlenecks, but they remain summaries of trajectories. A proof-backed lesson can be reliable as a method while still erasing local branch structure from the underlying episodes. The original trajectories and verification receipts therefore remain addressable whenever a later audit needs to reconstruct what was actually checked.

5 Counterexample-first search

Before expensive proof search, the system attempts cheap falsification: exact finite enumeration where available, randomized or property-based testing, boundary cases, computer algebra, SAT/SMT/model finding, adversarial parameter search and retrieval of stronger known parent results. These tools are especially useful because a single counterexample can close a large proof branch.

The authority rule is deliberately asymmetric. A counterexample may refute a universal conjecture; absence of a counterexample does not prove it. If E_n denotes successful testing on n cases, then

$$E_n \not\Rightarrow \forall x\, P(x)$$

for every finite n unless an independent theorem reduces the universal domain to those cases.

6 Formalization is a separate proof obligation

A proof assistant checks a formal proposition, not the researcher's informal intention. Consequently an accepted proof can coexist with a specification error. AlphaProof itself depends on formal problem statements and reports substantial formalization infrastructure [2]; this is not an incidental preprocessing detail but part of the epistemic chain.

We therefore bind an informal claim hash h_I to a formal statement hash h_F with a *formalization witness*. The strict witness records quantifier order, domains and codomains, assumptions, round-trip paraphrase, positive and negative examples, boundary cases and independent review. A proof receipt is accepted for promotion only when its theorem-statement hash equals h_F.

7 Proof assurance and verifier trust

Formal verification sharply reduces one class of error but introduces a trust boundary. Lean uses a small kernel to check proof terms; tactic bugs do not normally threaten kernel soundness because tactics must produce terms accepted by the kernel [5]. Finished developments must nevertheless audit their transitive axioms. Lean's documentation states that `sorryAx`, used by `sorry`, can prove arbitrary propositions and is not intended in finished proofs [6].

The strict profile records a proof receipt containing theorem and source hashes, checker identity and version, transitive axioms, and an independent recheck where supported. It rejects `sorryAx`, unregistered custom axioms and, when independent kernel-level assurance is required, proof paths that depend on compiler/native trust. This last condition is motivated by the fact that formal verification itself has an implementation trust surface: Lean 4.32.1, released in July 2026, fixed a kernel soundness bug and noted that the recommended separate `comparator` path for potentially dishonest proofs was not affected [7].

Proposition 2 (Generator-error containment)**.** *Let G be any proposal generator, K a sound proof checker for logic L, and U the canonical update rule. Suppose*

$$U(T, p) = \text{promote} \Longrightarrow K(p, T) = \text{accept}.$$

Then, relative to the soundness of K and the registered axioms, an invalid theorem cannot enter canonical truth state solely because G generated a plausible but invalid proof step.

Proof. The generator has no canonical write authority. Every promoted theorem must have an accepted proof artifact. By soundness of K, acceptance implies derivability of T from the registered assumptions. Therefore an invalid proposal generated by G is either rejected or would require failure of a trust assumption outside the generator itself. □ □

Proposition 3 (Verified checkpointing converts local generator error into search cost)**.** *Assume a final theorem depends only on lemmas admitted to a persistent DAG after independent proof checking. Then the per-proposal probability that an LLM suggests an invalid local step does not multiply directly into the logical validity of the accepted theorem. Invalid proposals instead increase rejected branches, latency or compute, unless a specification or verifier-trust assumption fails.*

Proof. An invalid proposal that fails checking is not added to the trusted dependency DAG, hence cannot serve as a premise of the accepted theorem. The final theorem's logical status depends on the accepted proof terms and checker assumptions, not on the number of invalid discarded proposals. □ □

This proposition does not make research easy. It changes the failure geometry. Long-horizon model error remains costly, and the hard unsolved problems become search, formalization, representation, novelty and trust rather than hidden survival of one invalid implication.

7.1 A long-horizon assurance decomposition

The difference between an unchecked derivation and a verifier-gated research process can be stated as a simple event bound. Let F be the event that a promoted result is false *as the intended mathematical claim.* Let E_{spec} denote failure of the informal–formal specification alignment gate, and let E_{trust} denote failure of the proof-checking trust chain, including unsound checker behaviour, unregistered axioms, incorrect dependency identity or artifact substitution. Assume canonical promotion is fail-closed: a result is promoted only when the alignment witness and the exact proof receipt pass their respective gates.

Proposition 4 (Assurance decomposition for a verifier-gated proof DAG)**.** *Under the fail-closed promotion rule above,*

$$F \subseteq E_{\text{spec}} \cup E_{\text{trust}}.$$

Consequently, for any probability measure describing the deployment environment,

$$\Pr(F) \leq \Pr(E_{\text{spec}}) + \Pr(E_{\text{trust}}).$$

In particular, local proposal errors made by the LLM do not appear as an additional additive term merely because the research horizon contains many generated steps, provided rejected proposals never enter the trusted proof DAG.

Proof. Consider a promoted result outside $E_{\text{spec}} \cup E_{\text{trust}}$. Since E_{spec} does not occur, the checked formal statement faithfully represents the intended claim under the declared alignment criterion. Since E_{trust} does not occur, the exact admitted proof artifact is valid under the registered assumptions and verifier semantics. Therefore the intended claim is not false, so F cannot occur. This proves the set inclusion. The probability inequality follows from the union bound. □ □

Remark 1. *This is an assurance decomposition, not a calibrated numerical guarantee. Estimating either term is a separate empirical problem. The important structural point is that, once every accepted dependency is verifier-gated, a thousand hallucinated candidate moves need not create a thousand hidden logical liabilities. They can create substantial search cost, and they can indirectly increase pressure on formalization or infrastructure, but they acquire truth authority only by passing the explicit gates.*

For novelty, a distinct error event E_{novel} must be tracked. A result promoted to the strongest new-mathematics candidate stage therefore has a broader research-level risk decomposition,

$$\begin{aligned}\Pr(F_{\text{research}}) \leq\ & \Pr(E_{\text{spec}}) + \Pr(E_{\text{trust}}) \\ & + \Pr(E_{\text{novel}}) + \Pr(E_{\text{value}}),\end{aligned}$$

where E_{value} denotes failure of the declared research-value review criterion. This second expression is likewise a union-bound decomposition, not an assumption that the events are independent.

8 Novelty is bounded and defeasible

A verified theorem is not automatically new. Newness requires a literature world, an equivalence notion and a cutoff. Define a bounded novelty certificate

$$N_t(T) = (C_{\leq t}, S, \nu, f(T), R),$$

where $C_{\leq t}$ is a named corpus snapshot, S the search routes, ν a normalization/equivalence procedure, $f(T)$ a theorem fingerprint and R the review record. Search routes should include exact wording, terminology variants, notation normalization, structural similarity, stronger-parent theorem retrieval, citation neighborhoods and multilingual/domain-synonym searches.

Proposition 5 (Novelty non-monotonicity)**.** *Let $C_t \subseteq C_{t+1}$ be literature corpora and define $\text{Novel}_C(T) = 1$ when no equivalent prior result is found in C. Then*

$$\text{Novel}_{C_t}(T) = 1$$

does not imply

$$\text{Novel}_{C_{t+1}}(T) = 1.$$

Proof. Choose a theorem T' equivalent to T such that $T' \notin C_t$ and $T' \in C_{t+1}$. The novelty predicate is true at t and false at $t+1$. □ □

Thus novelty authority is defeasible even when proof validity of the fixed statement is unchanged. The licensed statement is "no equivalent result was found in the registered novelty world at cutoff t," not "global novelty has been proved." This distinction also explains why rediscovery can be scientifically informative without being mislabelled as new mathematics.

9 The research-value gate

Truth and bounded novelty still do not imply importance. The system therefore separately records research-value judgments: generality, nontriviality, compression of many cases, a new invariant or representation, connection to an open problem, a stronger bound, a new proof technique, explanatory value, algorithmic utility, downstream theorem yield and expert interest. These signals rank what to pursue; they never compensate for missing proof authority.

10 Search policy: from plausible continuation to obstruction removal

At each point the system identifies a minimal or approximate set of unresolved blockers intersecting viable proof routes: a proof-theoretic analogue of an epistemic cut set. Candidate actions are selected for their expected ability to remove blockers or split ambiguity. High-value actions include proving a reusable bottleneck lemma, finding a counterexample that kills a family of conjectures, changing coordinates, importing a structurally analogous invariant, weakening an overstrong conjecture, or locating a known parent theorem.

This architecture aligns with recent successful discovery systems. FunSearch couples a pretrained model to systematic evaluation and evolutionary selection, producing new constructions and algorithms [1]. AlphaEvolve expands this evaluator-guided search pattern to scientific and algorithmic discovery [3]. AlphaProof treats formal proof search as sequential decision making

inside Lean [2]. Large-scale AlphaEvolve experiments report both rediscovery of known best solutions and improvements on several mathematical problems, highlighting why discovery and novelty classification must be separated [4]. The common lesson is that generation becomes more reliable when downstream state transitions are controlled by evaluators.

11 RAKL integration

The v3 framework makes this specialization explicit. The mathematical state $\mathfrak{M}_t$ is one authority-owning view inside a broader persistent RAKL state whose episode ledger, lessons, tools, failure view and method variants can influence retrieval and planning without inheriting theorem authority. Mathematical proof receipts remain owned by the assurance layer; a unified substrate edge can show that a lesson was derived from a proof episode or that a tool succeeded on a task, but that lineage edge does not replace the proof artifact.

The mathematical assurance layer is a specialization of existing RAKL principles rather than a competing architecture. The proposal/authority firewall becomes the theorem-promotion gate. Epistemic cut sets become unresolved proof or novelty blockers. Constructive invention proposes conjectures, lemmas, definitions and representations. Structural witnesses support notation-independent analogy and prior-art search. Negative history stores refuted conjectures and failed proof attempts. The authority poset is extended by specification, truth, novelty, value and trust coordinates.

The current reference implementation introduces:

- `src/rakl/math_research_assurance.py`: typed records, proof and novelty audits, and promotion states;
- `skills/rakl-core/workflows/mathematical-research.md`: the operational research workflow;
- `tests/test_math_research_assurance.py`: invariants including no promotion from computation, no `sorryAx`, no proof-to-novelty escalation, and novelty revocation without truth revocation.

The promotion sequence is intentionally conservative:

$$\begin{aligned}\texttt{CONJECTURE} \to \texttt{FORMALIZED\_UNPROVEN} \to \texttt{MACHINE\_PROVEN} \\ \to \texttt{BOUNDED\_NOVEL\_RESULT} \to \texttt{NEW\_MATHEMATICS\_CANDIDATE}.\end{aligned}$$

No step is automatic.

12 Typed discovery search and reference implementation

The assurance layer controls promotion, but a research system also needs a disciplined way to choose what to try next. We therefore couple theorem assurance to a typed problem-solving transition system. A problem state is represented as

$$S = (\sigma, F, O, R, B, H, \tau),$$

where σ is a problem signature, F the current planning facts, O open obligations, R registered representations, B explicit obstructions, H operator history and τ terminal status. A reusable research operator carries typed preconditions, state effects, targeted blockers, cost, verification debt, boundary risk and failure modes.

The operational structure is deliberately not claimed to be one global lattice. Method composition is partial and generally non-commutative. For example, formal proof search may

require a formal-statement fact produced by a prior formalization operator, so the reverse ordering is not licensed. Local lattices or partial orders are used only for relations whose meet/join or comparison obligations are separately established.

The reference planner performs bounded best-first search over candidate operator paths. For a path π, the current transparent planning score is

$$J(\pi) = C(\pi) + 2D(\pi) + 2R(\pi) + |B_{\text{remain}}| - 3|B_{\text{relieved}}|,$$

where C is modeled cost, D verification debt and R boundary risk. This score is not a probability of correctness. It only prioritizes candidate research actions according to explicit blockers and costs.

Reusable strategy motifs sit one level above primitive operators. A motif is an explicitly typed operator sequence. One motif changes representation, introduces an auxiliary object and then searches for an invariant. Another attempts cheap counterexample search before proof search and exact verification. Motifs are instantiated only as far as their operator preconditions remain satisfied. They are reusable search priors, not proof authority.

Crucially, planning transitions have no terminal write authority. A symbolic operator may propose a representation, lemma, auxiliary object or proof artifact, but only a separately verified terminal certificate may close a problem as a proof, counterexample, relative-independence result, reformulation or partial result. The planner therefore cannot turn path completion into theorem truth by construction.

Long-horizon reasoning is externalized into a persistent typed proof DAG. Nodes can represent conjectures, lemmas, theorems, definitions, computations, representations, imported results and counterexamples. Dependency cycles in proof-bearing relations are rejected. A lemma becomes a persistent verified checkpoint only when an exact statement-bound proof receipt passes the strict trust audit; refuted conjectures remain explicit nodes rather than being deleted. This is the executable mechanism behind the claim that rejected local generations become search cost and negative history instead of hidden logical debt.

The implementation surfaces are:

- `src/rakl/problem_solving_algebra.py`: typed problem signatures, obstructions, partial operators, bounded path search and certificate-gated terminal closure;
- `src/rakl/strategy_motifs.py`: reusable typed operator sequences with partial instantiation;
- `src/rakl/proof_dag.py`: persistent proof-state DAG, dependency-cycle checks, exact verified checkpoints and preserved refutations;
- `src/rakl/math_research_runtime.py`: compilation of theorem-assurance state into blockers and candidate paths;
- `src/rakl/math_research_assurance.py`: specification, proof, trust, novelty and value promotion gates;
- `src/rakl/math_research_api.py`: stable public facade for the reference profile;
- `benchmarks/math_research_assurance/tasks_v0.json`: hostile conformance worlds;
- `docs/MATH_RESEARCH_QUICKSTART.md`: executable usage path.

Exact release identity. Machine-readable receipts bind evaluated artifacts to exact release identities and their recorded test evidence; this stable manuscript source does not hard-code a Git subject or an aggregate repository test count. The hostile packet checks, among other cases, that finite computation is not promoted to proof, a proof of a neighbouring formal statement is rejected, `sorryAx` and unregistered custom axioms are blocked, missing isolated rechecking is detected, proof does not mint novelty, rediscovery is not called new mathematics, and research value cannot compensate for missing assurance coordinates. Additional unit tests bind proof source/checker identities, reject malformed prior-art receipts, reject proof-dependency cycles, exercise partial/non-commutative operator composition and require exact receipts before a proof-DAG checkpoint can become verified. These are implementation-conformance results, not evidence of autonomous-discovery superiority.

Usability boundary. The reference runtime is ready to be used as an auditable planning and promotion layer: users can construct a problem signature and claim record, inspect explicit blockers, obtain candidate operator paths or reusable motifs, maintain verified lemma checkpoints, attach formalization/proof/novelty receipts and query the resulting authority stage. It does not include a universal theorem prover, a complete mathematical literature index or a guarantee that its current operator basis is discovery-optimal. External LLMs, CAS/SAT/SMT tools, proof assistants and literature systems remain replaceable proposal/evaluation backends behind the typed contracts.

13 Vector saturation and invention boundaries for mathematical research

RAKL v3 separates several reasons a mathematical search may appear exhausted. Literature/knowledge routes, proof operators, recurring experience patterns, known obstructions, structural relations, successful composition paths and meta-methods can become locally flat at different times. Let

$$\mathbf{s}_t^{\mathrm{math}} = (s_K, s_O, s_E, s_B, s_R, s_P, s_M)$$

be the corresponding scoped saturation vector for a registered mathematical project. Flatness of s_P says that declared proof-path route families added no retained path novelty in the active window; it does not prove the theorem unprovable. Flatness of s_K says the registered novelty/literature routes were locally flat; it does not prove global novelty. Flatness of all declared coordinates remains bounded by their route families, cutoffs and resource envelope.

This distinction also gates invention. Being stuck does not establish a missing proof operator or representation. Escalation to representation/operator invention requires repeated stable residuals, exclusion of ordinary failures, bounded-flat relevant knowledge/operator/path routes, exhausted registered transfer routes and explicit evidence of a basis gap. Candidate invention then remains proposal-side until independent proof or protected assurance shows that the new operation closes the registered gap without violating theorem or verifier-trust invariants.

The architecture therefore permits aggressive creative search while making a negative claim difficult to mint. “No proof found”, “no known method retrieved”, “representation gap suspected” and “impossibility proved” are four different states with different evidence obligations.

14 Preregistered evaluation

Exercise benchmarks remain useful but insufficient. We preregister ten classes of evaluation:

1. hidden-theorem proof search with fixed formal statements;

2. false conjectures with difficult late counterexamples;

3. autoformalization traps in which a nearby but unintended theorem is easy to prove;
4. long proof DAGs requiring reusable intermediate lemmas;
5. rediscovery versus novelty under notation and terminology changes;
6. detection of stronger known parent theorems;
7. novelty evaluation against withheld or post-cutoff corpora;
8. open construction/optimization problems with executable evaluators;
9. trust attacks using `sorry`, custom axioms, native/compiler trust or artifact substitution;
10. blinded research-value ranking by expert mathematicians.

Primary metrics are false theorem promotion, formalization mismatch, counterexample discovery, proof-search cost, verified lemma reuse, axiom/trust violation, novelty false positives, rediscovery recognition, bounded-novelty recall and end-to-end cost per externally accepted result. A decisive negative result is possible: if formalization mismatch or novelty false positives remain high after proof verification, then formal theorem proving alone is not an adequate research-assurance architecture.

15 Limitations

The framework cannot prove that a finite novelty search is globally complete. It cannot remove the need for mathematical judgement about interestingness. It cannot guarantee that formalization faithfully captures intention without some semantic review. It inherits trust from the proof assistant, imported libraries and checker deployment. Search may remain computationally prohibitive on genuinely hard open problems. Finally, a system can be perfectly reliable at rejecting bad claims while discovering nothing useful; reliability and creative productivity must therefore be evaluated separately.

16 Conclusion

V3 interpretation. The framework can now accumulate and reuse mathematical task experience while keeping the proof/novelty/value authority coordinates separate. This strengthens the practical search architecture but not the theorem claim: future success may improve because the system remembers better routes, and future failure may become less repetitive because it remembers better boundaries, yet every promoted theorem still depends on the registered proof and verifier-trust chain. The scientific question left open is whether this experience substrate improves fresh mathematical-research outcomes under matched resources without increasing specification, novelty or trust failures.

The practical target for AI mathematics should not be an unconstrained model that writes longer proofs. It should be a system in which creative generation can be extremely aggressive because authority is extremely conservative. Formal verification changes the long-horizon error problem by turning many hallucinated steps into failed search rather than accepted mathematics. But proof correctness solves only one axis. Research-grade mathematical discovery also requires statement alignment, novelty evidence, research-value judgement and a hardened verifier trust chain. The design principle is therefore

scale creativity freely; scale mathematical authority only through explicit assurance.

Code, materials and AI-use disclosure

The public research artifact is maintained at `https://github.com/SzeChunYiu/RAKL`. The release package binds the manuscript to an exact Git subject, passing-test count, hostile assurance benchmark receipt and publication-artifact manifest. The repository contains the typed mathematical-research assurance runtime, benchmark task packet, workflow specification and release builder used by this paper. Language models were used as research, coding and drafting tools; they are not authors and their generated proposals receive no mathematical authority without the verification and governance steps described in the paper. Personal author metadata, funding and competing-interest declarations are supplied separately at submission and are not inferred by the build system.

References

[1] B. Romera-Paredes et al., "Mathematical discoveries from program search with large language models," *Nature* 625, 468–475 (2024). doi:10.1038/s41586-023-06924-6.

[2] T. Hubert et al., "Olympiad-level formal mathematical reasoning with reinforcement learning," *Nature* 651, 607–613 (2026). doi:10.1038/s41586-025-09833-y.

[3] A. Novikov et al., "AlphaEvolve: A coding agent for scientific and algorithmic discovery," arXiv:2506.13131 (2025).

[4] B. Georgiev, J. Gómez-Serrano, T. Tao and A. Z. Wagner, "Mathematical exploration and discovery at scale," arXiv:2511.02864 (2025).

[5] Lean Project, *The Lean Language Reference*, current 2026 documentation, section on the kernel and proof validation.

[6] Lean Project, *The Lean Language Reference: Axioms*, current 2026 documentation.

[7] Lean Project, "Lean 4.32.1 (2026-07-22)," release notes, 2026.

[8] E. Jiang et al., "From Solvers to Research: Large Language Model-Driven Formal Mathematics at the Research Frontier," arXiv:2607.07779 (2026).

[9] L. Pu et al., "MA-ProofBench: A Two-Tiered Evaluation of LLMs for Theorem Proving in Mathematical Analysis," arXiv:2606.13782 (2026).